

Hashing and Hash Functions

Why Hashing?

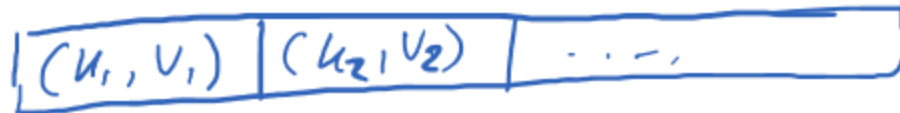
(key, value) pairs

(Student id) $\rightarrow$ (Student record)

Goal: efficiently insert, look up, and delete pairs

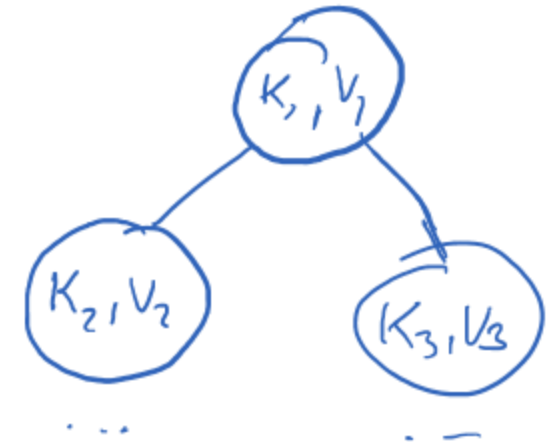
array, linked list

$O(N)$

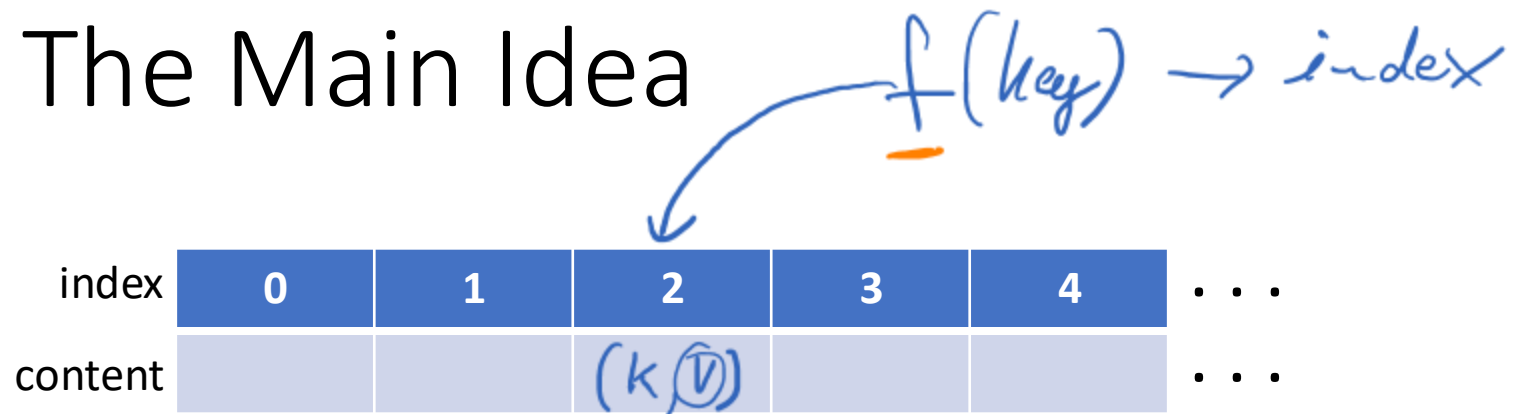


$N \Rightarrow \# \text{key, value pairs stored}$

$O(\log N)$



Hashing: The Main Idea



hash functions:



location: valid hash index
 $\geq 0, < \text{array.length}$

Ideal Hashing

Assume:

- We want to store 100 student records
- We use an array of size 100
- We use student IDs as keys

student IDs: 11 000, 11 001, 11 002, ..., 11 099

hash function: `int hash(int key) {`
 `return key - 11 000;`
 `// or: return key % 11 000;`
 `}`

Perfect Hash Function

A perfect hash function maps every key to a **unique** hash index.

K : key-type

V : value type

Operations

field: $V[]$ array

```
void insert(K key, V value) {  
    array[hash(key)] = value;  
}
```

```
V lookup(K key) {  
    return array[hash(key)];  
}
```

```
void remove(K key) {  
    array[hash(key)] = null;  
}
```

Real World: UW Student IDs

10 digit numbers: 9021453190
 9033879101
 9024357190

collision

Can we find a perfect hash function?



Properties of a “Good” Hash Function

1. must be deterministic
2. should achieve uniform distribution across output range
3. should minimize collisions
4. should be fast and easy to compute

Java API for Hash Functions

int hashCode() method:

- instance method of **Object** type
- Java has implementations for built-in data types (String, Double, etc.)
- we can override it for our own data types

Steps for using int hashCode():

1. call hashCode() on key object
2. convert result (in range of integer) to valid hash index (abs, modulo)

$$\text{Math.abs}(\text{key.hashCode()}) \% \text{array.length}$$

↳ valid hash index